

COMPENDIO TÁCTICO: BACKTRACKING

Búsqueda con Retroceso, Podas y Espacio de Soluciones (Edición Completa)

1. Estructura y Conceptos Fundamentales

A diferencia de la Fuerza Bruta que genera todas las combinaciones y luego filtra, la **Búsqueda con Retroceso (Backtracking)** forma las soluciones de manera progresiva, elemento a elemento. Si detecta que la tupla parcial actual $(x_1, \dots, x_i)$ no puede conducir a una solución válida, **corta esa rama del árbol** inmediatamente y retrocede.

El Espacio de Búsqueda y las Tuplas

- **El Espacio de Búsqueda (S):** Es el conjunto de todas las combinaciones posibles. Matemáticamente: $S = \{(x_1, \dots, x_n) \mid x_i \in C_i\}$.
- **Tuplas de Tamaño Fijo vs Variable:** Es la decisión arquitectónica más importante.
 - *Fijo:* El vector siempre tiene tamaño N . Los valores x_i suelen ser decisiones binarias (ej. 1 = lo meto en la mochila, 0 = no lo meto). Genera un árbol binario.
 - *Variable:* El vector va creciendo. Cada x_i representa el *índice* de un elemento elegido (ej. $x_1 =$ índice del primer objeto). Genera árboles más anchos.

2. Metodología de Resolución: Los 5 Pasos Innegociables

Todo ejercicio de examen te pide definir la representación, el árbol y los predicados. Debes contestar siempre con esta estructura exacta:

- **PASO 1: Representación de la Solución (La Tupla)**
 - *La Lógica:* Decidir si usas tamaño fijo o variable. Qué representa el vector X y qué representa el índice i .
 - *Plantilla:* Una solución se representa como una tupla $(x_1, \dots, x_n)$ donde cada x_i representa [el color del nodo i / si cojo o no el objeto i / la posición del guardia i].
- **PASO 2: Restricciones Explícitas (C_i)**
 - *La Lógica:* Describen el dominio matemático puro de cada variable x_i , es decir, qué valores sueltos puede tomar sin mirar a sus vecinos.
 - *Ejemplo:* $x_i \in \{0, 1\}$ (Mochila binaria) o $x_i \in \{1, \dots, N\}$ (Coloreado de grafos).
- **PASO 3: Restricciones Implícitas (Las Reglas del Juego)**
 - *La Lógica:* Describen las relaciones que **deben cumplirse entre las variables** x_i para que el resultado tenga sentido en el mundo real.
 - *Ejemplos:* "La suma de los pesos no puede superar C ($\sum p_i x_i \leq C$).^o "Dos nodos adyacentes no pueden tener el mismo color ($x_i \neq x_j$)".
- **PASO 4: Predicados Acotadores / La Poda ($P_i(x)$)**
 - *La Lógica:* Es el "freno de emergencia". Es una función booleana que evalúa la tupla parcial $(x_1, \dots, x_i)$ y devuelve FALSO si es imposible que lleguemos a una solución desde aquí.
 - *Plantilla Teórica:* $P_i(x) : C_1 \times \dots \times C_i \rightarrow \{\text{true}, \text{false}\}$.
- **PASO 5: Predicado de Solución ($R_i(x)$)**
 - *La Lógica:* Condición que determina si la tupla parcial ha alcanzado el final y es una solución completa y válida que debemos imprimir/guardar.
 - *Ejemplos:* $i == n$ (hemos procesado todos los elementos) o $\sum a_{x_j} == S$ (hemos alcanzado la suma exacta).

3. El Esqueleto Universal de Backtracking

Apréndete este esquema de memoria. Funciona para el 90 % de los problemas.

Función Recursiva Base

```
// Llamada inicial: backtracking(x, 1)
void backtracking(tupla x, int i) {
    // 1. G_i(x): Generamos los candidatos posibles para la posición i
    for each nodo en G_i(x) {

        x[i] = nodo; // Probamos el candidato

        // 2. Acotador (P_i): ¿Tiene sentido seguir por aquí?
        if ( P_i(x) == true ) {

            // 3. Solución (R_i): ¿Hemos terminado el problema?
            if ( R_i(x) == true ) {
                escribir_solucion(x, i);
            }

            // 4. Avance: Si no hemos terminado y es válido, seguimos bajando
            if ( i < n ) {
                backtracking(x, i + 1);
            }
        }
    }
}
```

4. Plantillas Mentales por Patrón

■ A. Patrón de Subconjuntos (Tamaño Fijo / Binario)

- *Cuándo usar:* Te dan un conjunto y tienes que decidir si coges o no cada elemento (ej. Mochila 0-1, Distancia de Hamming).
- *El For:* for each decision in {1, 0}

■ B. Patrón de Tableros y Asignaciones

- *Cuándo usar:* N-Reinas, Sudoku, Asignar Colores a Nodos (Ej. 4), Colocar Guardias en matriz (Ej. 5).
- *El For:* for each valor in {1 ... max_opciones} (ej. de 1 a N columnas, o de 1 a C colores).
- *El Acotador:* Suele requerir un bucle auxiliar que compruebe que el valor actual no "choca" (diagonales, aristas) con ninguno de los $x_1 \dots x_{i-1}$ ya puestos.

■ C. Patrón de Permutaciones / Rutas

- *Cuándo usar:* El orden importa. Ciclos Hamiltonianos (Ej. 6), Expresiones Aritméticas (Ej. 2).
- *El For:* for each nodo in {1 ... N}
- *El Acotador:* El nodo actual NO puede estar ya en la tupla (requiere un vector auxiliar usados[nodo] == false). Además, debe existir una arista válida desde x_{i-1} hasta x_i .

■ D. Patrón de Optimización (Maximizar/Minimizar)

- *Modificación estructural:* La función debe recibir y arrastrar una variable global mejor_solucion. Solo se imprime al terminar todo el árbol.
- *Poda extra (P₂):* Se calcula una cota teórica (ej. con un algoritmo Voraz). Si cota_actual < mejor_beneficio, podamos la rama entera.

5. Análisis de Problemas Canónicos (Teoría y Boletín)

A. Suma de Subconjuntos (Teoría)

- **Ficha de Sustitución:** *Patrón:* Variable. *Objetivo:* Suma S .
- **Representación:** $(x_1, \dots, x_i)$ donde x_j son **índices** de los elementos escogidos.
- **Restricciones Explícitas:** $x_j \in \{1, \dots, n\}$.
- **Restricciones Implícitas:** 1) Orden estricto para evitar repetidos: $x_j < x_{j+1}$. 2) La suma debe ser S : $\sum a_{x_j} = S$.
- **Acotador** $P_i(x)$: $\sum_{j=1}^i a_{x_j} \leq S$ (Si ya me he pasado de la suma objetivo, corto).

B. El Problema de la Mochila 0-1 (Teoría)

- **Ficha de Sustitución:** *Patrón:* Fijo Binario. *Variables:* $x_i \in \{0, 1\}$.
- **Restricciones Implícitas:** $\sum p_i x_i \leq C$ (No reventar la mochila).
- **Acotadores (P_1 y P_2):** *Ojo de examen.*
 - P_1 (Viabilidad): `totPeso <= C`.
 - P_2 (Optimización): `cota() > mejor.ben`. Calculamos la cota con algoritmo Voraz fraccionario. Si incluso metiendo fracciones no supero la mejor solución actual, corto la rama. **Aviso:** Exige ordenar antes los objetos por beneficio/peso.

C. Coloreado de Grafos para la Escuela (Boletín Ej. 4)

- **Ficha de Sustitución:** *Patrón:* Tableros/Asignaciones (Optimización). *Nodos:* Alumnos/Vértices. *Colores:* $1 \dots C_{max}$.
- **Representación:** $(x_1, \dots, x_n)$ donde x_i es el color asignado al vértice i .
- **Acotador $P_i(x)$:** Comprobar que para todo vecino j del vértice i que ya haya sido coloreado ($j < i$), se cumpla $x_i \neq x_j$.
- **Optimización:** El acotador también debe incluir: si el color actual supera el "mínimo de colores" encontrado hasta ahora, cortar rama.

D. Guardias del Sultán en Matriz NxN (Boletín Ej. 5)

Es una variante camuflada de las N-Reinas, introduciendo "muros" obstáculos que bloquean la visión.

- **Representación:** $(x_1, \dots, x_n)$ donde x_i representa la **columna** en la que coloco al guardia de la fila i . (Si una fila puede tener varios guardias por culpa de los muros, se puede usar matriz binaria $N \times N$, $x_{ij} \in \{0, 1\}$).
- **Acotador $P_i(x)$:** "No se pueden ver". Desde la posición actual, lanzar rayos simulados en filas, columnas y diagonales hacia atrás. Si choco con un muro, el rayo se detiene (es seguro). Si choco con otro guardia antes que con un muro, **Podar** (devuelve Falso).

E. Distancia de Hamming desde cadena "Nula" (Boletín Ej. 1)

- **Análisis rápido:** Distancia h desde cadena de ceros $\Rightarrow$ La cadena debe tener exactamente h unos.
- **Representación:** $(x_1, \dots, x_n)$, donde $x_i \in \{0, 1\}$.
- **Acotador $P_i(x)$:** Llevo la cuenta de los unos (**suma**). Si **suma > h**, podar. Si me quedan posiciones por rellenar ($n - i$) y aunque las ponga todas a 1 no llego a h (**suma + (n-i) < h**), podar también.

F. Cestas de Frutas del Sr. Martin (Boletín Ej. 3)

- **Ficha de Sustitución:** *Patrón:* A (Subconjuntos / Tamaño Fijo). *Variables:* $x_i \in \{0, 1\}$.
- **Representación:** Una solución es una tupla de tamaño fijo n : $(x_1, \dots, x_n)$ donde $x_i = 1$ si la fruta i entra en la cesta, y 0 si no.
- **Restricciones Explícitas:** $x_i \in \{0, 1\}$ para $1 \leq i \leq n$.
- **Restricciones Implícitas:** El peso total de la cesta debe estar acotado: $200 \leq \sum_{i=1}^n x_i \cdot p_i \leq 1000$.
- **Acotador $P_i(x)$:** "No reventar la cesta". Llevamos el peso acumulado. Si **peso_actual > 1000**, podamos la rama (devolvemos falso).
- **Predicado Solución R_i :** Llegamos al final del árbol ($i == n$) **Y** además el peso alcanza el mínimo exigido (**peso_actual >= 200**).

G. Ciclos Hamiltonianos sin Aristas Prohibidas (Boletín Ej. 6)

- **Ficha de Sustitución:** *Patrón:* C (Rutas/Permutaciones). *Variables:* $x_i \in \{1 \dots N\}$.
- **Representación:** Tupla de tamaño fijo N : $(x_1, \dots, x_N)$ que representa la secuencia exacta de vértices visitados.
- **Restricciones Explícitas:** $x_i \in \{1, \dots, N\}$.
- **Restricciones Implícitas:** 1) No hay vértices repetidos ($x_i \neq x_j$). 2) Existe una arista permitida entre x_{i-1} y x_i . 3) Existe arista de cierre entre x_N y x_1 . 4) Para no contar repetidos cíclicos: $x_1 = 1$ y $x_2 < x_N$.
- **Acotador $P_i(x)$:** Comprobar que v no esté en el array de **visitados**, y comprobar que la arista (x_{i-1}, v) sea válida (no prohibida).
- **Predicado Solución R_i :** Cuando $i == N$, solo falta comprobar si la arista de cierre (x_N, x_1) es válida.

H. Expresiones Aritméticas Permutadas (Boletín Ej. 2)

- **Ficha de Sustitución:** *Patrón:* C (Permutaciones) + Decisiones Múltiples.
- **Representación:** $(x_1, o_1, x_2, o_2, \dots, x_n)$ donde $x_i \in \{1 \dots n\}$ son los índices de los números, y $o_i \in \{+, *\}$ son los operadores.
- **Restricciones Explícitas:** $x_i \in \{1 \dots n\}$ y $o_i \in \{+, *\}$.
- **Restricciones Implícitas:** Es una permutación (no se repiten números). El resultado matemático final debe ser exactamente E .
- **Acotador** $P_i(x)$: 1) Para números: Verificar que el índice no esté en usados. 2) Para acumulación: Si todos los números son > 0 , el valor parcial nunca decrece. Si `valor_parcial > E`, podamos la rama.
- **Predicado Solución** R_i : $i == n$ y `valor_parcial == E`.

6. Escudo Teórico y Tácticas Avanzadas

Técnica Meet-in-the-Middle (Salvar el Coste Temporal)

Si un problema de Backtracking es $\mathcal{O}(2^n)$ (fuerza bruta de subconjuntos) y n es relativamente grande (ej. $n = 40$), el árbol explota. La técnica de "encontrarse en el medio" salva la vida:

1. **Dividir:** Se parte la lista original en dos mitades A y B de tamaño $\approx n/2$.
2. **Generar:** Se aplican los subconjuntos por separado, generando las listas de resultados S_A y S_B . Coste $\mathcal{O}(2^{n/2})$.
3. **Ordenar y Combinar:** Ordenas las listas y buscas parejas (un elemento de S_A y otro de S_B) que cumplan la condición global (ej. sumen S). Coste total reducido a $\mathcal{O}(n \cdot 2^{n/2})$.

7. Diccionario Rápido de Mapeo en Examen

- Si dice *Combinaciones / Suma igual a...*: $\Rightarrow$ **Subconjuntos (Variable o Fijo)**. Generas árbol, tu acotador vigila que no te pases de la suma total.
- Si dice *No pueden verse / No pueden tocarse / Grafo conexo*: $\Rightarrow$ **Tableros/Asignaciones**. Tu acotador recorre el historial $1 \dots i - 1$ comprobando distancias o aristas.
- Si dice *Ciclo / Permutación de los N dados*: $\Rightarrow$ **Rutas**. Usa vector fijo de $1..N$ pero con control de "visitados".
- Si dice *Mínimo número de colores / Máximo beneficio*: $\Rightarrow$ **Optimización**. Necesitas variable global `mejor_solucion` y calcular Cotas Voraces obligatoriamente para justificar la poda.

8. Análisis y Código de Problemas Canónicos (Teoría y Boletín)

Esta sección contiene la instanciación exacta de los 5 pasos, el análisis riguroso del Árbol de Búsqueda (tamaño y topología) y el Chasis Universal para todos los problemas.

A. Suma de Subconjuntos (Teoría)

- **Ficha de Sustitución:** *Patrón:* Variable. *Objetivo:* Suma S .
- **Representación:** $(x_1, \dots, x_i)$ donde x_j son **índices** de los elementos escogidos.
- **Árbol y Tamaño del Espacio (Paso 2b):** Es un árbol n -ario de profundidad variable. En cada nivel i , ramifica hacia los índices mayores que x_{i-1} . El tamaño máximo del espacio de soluciones (combinatoria de coger o no coger) equivale a 2^n hojas. Complejidad teórica en el peor caso: $\mathcal{O}(2^n)$.
- **Restricciones Explícitas:** $x_j \in \{1, \dots, n\}$.
- **Restricciones Implícitas:** 1) Orden estricto para evitar repetidos: $x_j < x_{j+1}$. 2) La suma debe ser S : $\sum a_{x_j} = S$.
- **Acotador** $P_i(x)$: $\sum_{j=1}^i a_{x_j} \leq S$. (Si me paso del objetivo, corto la rama inmediatamente).
- **Variables de Estado Adicionales:** `suma_actual` (pasada por valor en la recursividad, no hace falta deshacerla).

Implementación: Suma Subconjuntos

```
void sumaSub(tupla x, int i, int suma_actual, tupla a, int S) {
    // Dominio: Empezamos a buscar a partir del índice del elemento anterior + 1
    int inicio = (i == 1) ? 1 : x[i-1] + 1;
    for (int nodo = inicio; nodo <= n; nodo++) {
        int nueva_suma = suma_actual + a[nodo];

        if (nueva_suma <= S) { // PODA (Freno de viabilidad)
            x[i] = nodo; // Guardamos el índice

            if (nueva_suma == S) { // META
                escribir_solucion(x, i); // i es el tamaño real actual de la tupla
            }
            if (i < n) {
                sumaSub(x, i + 1, nueva_suma, a, S); // Profundizamos
            }
        }
    }
}
```

B. El Problema de la Mochila 0-1 (Teoría)

- **Ficha de Sustitución:** *Patrón:* Fijo Binario. *Variables:* $x_i \in \{0,1\}$.
- **Árbol y Tamaño del Espacio (Paso 2b):** Árbol binario puro de profundidad exacta n . En cada nivel hay 2 opciones (1=meter, 0=no meter). El número de nodos generados en fuerza bruta es $2^{n+1} - 1$, con un tamaño del espacio de búsqueda de 2^n soluciones posibles.
- **Restricciones Implícitas:** $\sum p_i x_i \leq C$ (No reventar la capacidad de la mochila).
- **Acotadores (P_1 y P_2):**
 - P_1 (Viabilidad): $\text{totPeso} \leq C$.
 - P_2 (Optimización): $\text{cota}(C - \text{nuevoPeso}, \text{nuevoBen}, i + 1) > \text{mejor_ben}$. Calculada mediante el algoritmo Voraz fraccionario. Exige ordenar previamente los objetos por densidad de valor (b_i/p_i) de mayor a menor.
- **Variables de Estado Adicionales:** Variables globales de récord (mejor_ben), y de arrastre (totPeso , totBen).

Implementación: Mochila 0-1

```
void mochila(tupla x, int i, int totPeso, int totBen) {
    for (int decision = 1; decision >= 0; decision--) { // Probamos meterlo y no meterlo
        int nuevoPeso = totPeso + p[i] * decision;
        int nuevoBen = totBen + b[i] * decision;

        // PODA COMBINADA: Que quepa físicamente Y que la cota supere el récord
        if (nuevoPeso <= C && cota(C - nuevoPeso, nuevoBen, i + 1) > mejor_ben) {
            x[i] = decision;

            if (i == n) { // META
                mejor_ben = nuevoBen;
                guardar_mejor_tupla(x);
            } else {
                mochila(x, i + 1, nuevoPeso, nuevoBen);
            }
        }
    }
}
```

C. Coloreado de Grafos para la Escuela (Boletín Ej. 4)

- **Ficha de Sustitución:** *Patrón:* Tableros/Asignaciones (Optimización). *Nodos:* Vértices $1..N$. *Colores:* $1 \dots C_{\max}$.
- **Representación:** $(x_1, \dots, x_N)$ donde x_i es el color asignado matemáticamente al vértice i .
- **Árbol y Tamaño del Espacio (Paso 2b):** Árbol n -ario de profundidad fija N (un nivel por vértice). Si el máximo de colores posibles es C , cada nodo ramifica en C opciones. Tamaño total del espacio de búsqueda: C^N . Complejidad

exponencial $\mathcal{O}(C^N)$.

- **Acotador $P_i(x)$ (Viabilidad):** $\forall j < i$, si $M[i][j] == 1 \implies x_i \neq x_j$. Dos nodos con arista no comparten color.
- **Acotador $P_i(x)$ (Optimización):** El máximo color usado en la rama actual (`max_color_actual`) debe ser estrictamente menor que la `mejor_solucion` guardada.
- **Variables de Estado Adicionales:** `max_color_actual` (para saber cuántos colores distintos llevamos en esta ruta sin tener que recalcularlo).

Implementación: Coloreado de Grafos (Optimización)

```
// Variable global: mejor_solucion inicializada a N+1 (peor escenario posible)
void colorear(tupla x, int i, matriz M, int max_color_actual) {
    // TRUCO DE ACELERACIÓN: Solo iterar hasta mejor_solucion - 1
    for (int color = 1; color < mejor_solucion; color++) {
        x[i] = color;
        int nuevo_max = max(max_color_actual, color);

        if (no_choca(x, M, i, color) == TRUE) { // PODA de adyacencia
            if (i == N) { // META
                mejor_solucion = nuevo_max; // Récord batido y actualizado
            } else {
                colorear(x, i + 1, M, nuevo_max);
            }
        }
    }
}
// no_choca: for(j=1..i-1) if(M[i][j]==1 && x[j]==color) return FALSE; return TRUE;
```

D. Guardias del Sultán en Matriz NxN (Boletín Ej. 5)

- **Ficha de Sustitución:** Problema Isomorfo de las N-Reinas. *Patrón:* Tableros.
- **Representación:** $(x_1, \dots, x_N)$ donde x_i representa la **columna** en la que coloco al guardia de la fila i . Esta representación elimina la necesidad de comprobar colisiones horizontales (filas).
- **Árbol y Tamaño del Espacio (Paso 2b):** Árbol n-ario de profundidad N (un nivel por fila). En cada fila, hay N opciones de columnas. Tamaño bruto del espacio: N^N . Si la implementación descarta columnas ya usadas automáticamente, el árbol se reduce a permutaciones: $N!$.
- **Acotador $P_i(x)$:** "No se pueden ver". Implica comprobar visualmente hacia las filas anteriores ($j < i$). Falla si hay otro guardia en la misma columna ($x_i == x_j$) o en la misma diagonal (distancia en filas igual a distancia en columnas: $|i - j| == |x_i - x_j|$). Al obligar a que no se vean, la excusa de usar "escudos" que da el enunciado es falsa (el escudo y el mirador chocarían entre sí).

Implementación: Guardias (N-Reinas)

```
void guardias(tupla x, int i) {
    for (int col = 1; col <= N; col++) {
        x[i] = col;

        bool seguro = TRUE;
        for (int j = 1; j < i; j++) { // PODA: Revisamos el historial hacia arriba
            if (x[j] == col || abs(j - i) == abs(x[j] - col)) {
                seguro = FALSE;
                break; // Freno inmediato
            }
        }

        if (seguro) {
            if (i == N) escribir_solucion(x); // META
            else guardias(x, i + 1);
        }
    }
}
```

E. Distancia de Hamming desde cadena "Nula" (Boletín Ej. 1)

- **Representación:** $(x_1, \dots, x_N)$, donde $x_i \in \{0, 1\}$.
- **Árbol y Tamaño del Espacio (Paso 2b):** Árbol binario puro de profundidad N . El tamaño del espacio de combinaciones posibles es exactamente 2^N .
- **Acotador $P_i(x)$:** Se aplica una doble poda predictiva:
 - *Poda por Exceso:* `unos_actuales <= h`. (No podemos pasarnos del límite de 1s).
 - *Poda por Defecto:* `unos_actuales + (N - i) >= h`. (Si quedan $N - i$ huecos libres y llenándolos todos de unos aún así no llegamos a h , podamos la rama por inservible).

Implementación: Distancia de Hamming

```
void hamming(tupla x, int i, int unos_actuales, int N, int h) {
    for (int bit = 0; bit <= 1; bit++) {
        int nuevos_unos = unos_actuales + bit;

        // PODA COMBINADA bidireccional (Exceso y Defecto)
        if (nuevos_unos <= h && nuevos_unos + (N - i) >= h) {
            x[i] = bit;
            if (i == N) {
                if (nuevos_unos == h) escribir_solucion(x); // META
            } else {
                hamming(x, i + 1, nuevos_unos, N, h);
            }
        }
    }
}
```

F. Cestas de Frutas del Sr. Martin (Boletín Ej. 3)

- **Ficha de Sustitución:** *Patrón:* A (Tamaño Fijo / Binario). *Variables:* $x_i \in \{0, 1\}$. Decidir para cada uno de los n tipos de fruta si se introduce 1 pieza o 0.
- **Árbol y Tamaño del Espacio (Paso 2b):** Árbol binario. Profundidad total n (una decisión por cada tipo de fruta). Tamaño del espacio de búsqueda total: 2^n cestas matemáticas posibles.
- **Restricciones Implícitas:** El peso total debe estar encajado: $200 \leq \sum_{i=1}^n x_i \cdot p_i \leq 1000$.
- **Acotador $P_i(x)$:** Poda obligatoria por límite superior (`peso_actual ≤ 1000`) durante la construcción.
- **Predicado Solución R_i :** Filtro estricto por límite inferior al llegar al final (`peso_actual ≥ 200`).
- **Variables de Estado Adicionales:** `peso_actual`. (Contadores globales `total_cestas` y `peso_total_historico` requeridos por el enunciado).

Implementación: Cestas de Fruta

```
// Variables Globales: int total_cestas = 0; int peso_total_historico = 0;
void cestas(tupla x, int i, int peso_actual) {
    for (int poner = 1; poner >= 0; poner--) {
        int nuevo_peso = peso_actual + p[i] * poner;

        if (nuevo_peso <= 1000) { // PODA: Evitar pasarse del kilo (1000g)
            x[i] = poner;

            if (i == n) { // META
                if (nuevo_peso >= 200) { // FILTRO FINAL: Alcanzar el mínimo (200g)
                    total_cestas++;
                    peso_total_historico += nuevo_peso;
                }
            } else {
                cestas(x, i + 1, nuevo_peso);
            }
        }
    }
}
```

G. Ciclos Hamiltonianos sin Aristas Prohibidas (Boletín Ej. 6)

- **Ficha de Sustitución:** *Patrón:* C (Rutas/Permutaciones). *Variables:* $x_i \in \{1 \dots N\}$.
- **Representación:** $(x_1, \dots, x_N)$ representa la cronología u orden exacto de los vértices visitados.
- **Árbol y Tamaño del Espacio (Paso 2b):** Árbol de permutaciones. Al estar el primer vértice fijo ($x_1 = 1$), la profundidad a explorar es $N - 1$. Cada nivel reduce sus opciones disponibles. El tamaño bruto del espacio de búsqueda es factorial: $(N - 1)!$. Complejidad: $\mathcal{O}(N!)$.
- **Restricciones Implícitas:** 1) $x_i \neq x_j$ (Array de usados). 2) Arista lícita entre x_{i-1} y x_i . 3) Arista de cierre para formar ciclo entre x_N y x_1 . 4) Ruptura de antimetría para no contar recorridos en sentido inverso: $x_1 = 1$ y obligatoriamente $x_2 < x_N$.
- **Variables de Estado Adicionales:** Array booleano usados[]. Requiere Backtrack explícito.

Implementación: Ciclos Hamiltonianos

```
// Arranque: x[1] = 1; usados[1] = TRUE; Llamada: hamiltoniano(x, 2)
void hamiltoniano(tupla x, int i) {
    for (int v = 2; v <= N; v++) { // El vértice 1 ya está fijado

        // PODA: El vértice debe estar libre Y la matriz de adyacencia debe tener arista
        if (usados[v] == FALSE && M[x[i-1]][v] == 1) {
            x[i] = v;
            usados[v] = TRUE; // APLICAR ESTADO

            if (i == N) { // META
                // Comprobar cierre geométrico del ciclo y matar simetría especular
                if (M[x[N]][x[1]] == 1 && x[2] < x[N]) {
                    contar_solucion();
                }
            } else {
                hamiltoniano(x, i + 1);
            }

            usados[v] = FALSE; // BACKTRACK: Deshacer estado (Crucial)
        }
    }
}
```

H. Expresiones Aritméticas Permutadas (Boletín Ej. 2)

- **Representación:** $(x_1, o_1, x_2, o_2, \dots, x_n)$ combinando en paralelo la permutación de los índices de los números $x_i \in \{1 \dots n\}$ y las operaciones elegidas $o_i \in \{+, *\}$.

- **Árbol y Tamaño del Espacio (Paso 2b):** Estructura de árbol mixto. Hay $n!$ permutaciones posibles para ordenar los números, y para cada hueco entre números hay 2 operaciones posibles (2^{n-1}). Tamaño monstruoso del espacio de soluciones: $n! \cdot 2^{n-1}$.
- **Acotador $P_i(x)$:** Si todos los números a_i del conjunto son enteros ≥ 0 , las sumas y multiplicaciones hacen que el valor sea monótonamente creciente. Si en algún paso `valor_parcial > E`, podemos aplicar una poda segura. Requiere control estricto del array `usados[]` para no repetir índices de números.

Implementación: Expresiones (Doble Decisión)

```
// La primera llamada carga el primer número elegido y empieza en i=2.
void expresiones(tupla x, tupla ops, int i, int valor_parcial) {
    for (int num_idx = 1; num_idx <= n; num_idx++) {
        if (usados[num_idx] == FALSE) {

            for (char operador in {'+', '*'}) { // Sub-bucle de operadores
                int nuevo_valor = (operador == '+') ?
                    valor_parcial + a[num_idx] : valor_parcial * a[num_idx];

                // PODA MONÓTONA: El valor nunca bajará. Si supero la Meta 'E', corto.
                if (nuevo_valor <= E) {
                    x[i] = num_idx;
                    ops[i-1] = operador;
                    usados[num_idx] = TRUE; // APLICAR ESTADO

                    if (i == n) {
                        if (nuevo_valor == E) escribir_expresion();
                    } else {
                        expresiones(x, ops, i + 1, nuevo_valor);
                    }
                    usados[num_idx] = FALSE; // BACKTRACK
                }
            }
        }
    }
}
```